

Supplementary Material for Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

S1. DETAILS OF THE ORIGINAL-EXECUTION CONNECTION

Earliest conflicting polka. Fix a height and a commit certificate for a nonempty value v in round r , and select two honest signers K of that certificate. Each locks v before it issues its precommit. The certificate can be assembled after these signers have advanced, so its observation time is irrelevant.

Suppose a higher-round polka for a different value exists, counting nil if it permits unlocking. Select the smallest round $s > r$ with such a polka and, within s , the earliest event τ at which its distinct prevotes exist. Its three voters include a member $m \in K$. Honest rounds increase, so the round- s prevote of m follows its round- r precommit, and without an unlocking justification m must prevote its locked value. Relocking on v in intervening rounds preserves this restriction. A conflicting justification from a round below s contradicts the choice of s , and one from round s must have existed before m cast the vote counted at τ , which contradicts the choice of τ . Evidence from above s cannot precede a later round- s vote. No such polka exists, and a higher-round conflicting commit would require one. Same-round conflicting commits require an honest double precommit. The argument rests on the stated lock-transition and round-monotonicity rules and does not need timely receipt of commit evidence.

Pairing, validation, and execution. Induct on candidate updates. Local construction pairs a body with its original parts, and remote completion rebuilds the body from checked parts. Moving Locked or Valid transfers both, and switching targets clears the old body. Historical installation leaves these active candidates alone. A nonempty precommit follows a matching polka. An honest prevote has a justification in actual proposal validation or an earlier lock, and tracing the latter lowers the lock round until it reaches validation. The final full-identity guard fixes the original transactions, height, time, and other application context before Finalize. Synchronization and replay supply the same original material, so same-height retries cannot introduce a second original body through the stable-hash cache within this call boundary.

Write sets. The proofs of the main text rely on the following write sets. A compensation decision writes the issuer reserve account, the obligation status, the Promise and coverage record (Paid and Remaining), grant usage (Spent), the public gap, the consumer's payment record (Pending and fee stages), reward and refund balances and fee outputs, the removal of the due-index entry, and the decision record with its decision-index and pending-representation entries. The execution of a source after compensation writes the repayment: the issuer reserve account, the obligation status (Recovered), the coverage record (Recovered), and grant usage (Spent). It leaves the decision record unchanged. A resubmission job writes the member's local outbox only. A representation command or batch writes the logical revision, the head command, the task or batch task with its per-output references, the queue entry, and the removal of pending-representation entries. Installation writes local block-store versions and its own progress cursor only. No representation or installation write belongs to the economic projection.

Convergence for a fixed target set. Fix an original body and a set of decided slots. The replacement message of each slot is a function of the network and the output, so any packaging replaces the same slots by the same messages. If the chameleon relation has a unique valid opening for each fixed message and commitment, as holds for correctly generated parameters with a public exponent coprime to the group order, each slot opening and each part opening is the unique solution of its relation. Part contexts depend on the height and the part index and are independent of the revision counter. Any grouping or order of representation commands that covers the set therefore ends in the same body and part openings. Single-item commands, one batch, and batches of intermediate sizes differ in command identities, revision numbers, and application hashes. The statement concerns final bytes and depends on parameter generation and encoding as stated.

Follower updates. For a follower that starts from the same local state, configuration, and cursor, equal fixed payment fields and authenticated results induce equal economic updates whenever both representations decode. The

mutable funding reference and opening select no Paid, Pending, or fee effect. Decisions are authenticated in full, and the execution-result commitments authenticate their effects and fee outputs. Economic observation thus stays separate from the independent verification of the latest mutable historical body.

Relations used in the analysis. Three relations compare different objects. Representation inertness compares the economic projection before and after one representation or installation step, and it projects away command identities, revision counters, and materialization tasks. Fixed-target convergence compares the final body and part openings that different groupings and orders of representation commands produce for one original body and one decided set. Replay equality fixes the entire original command sequence and its block contexts, and deterministic execution then reproduces each application hash. Cross-history comparisons fix the economic commands, public times, and historical target coordinates, so projection equality does not identify entire application states and does not permit deleting or renumbering blocks. None of the three relations compares histories that order parent arrivals and compensations differently; the delay-neutrality theorem of the main text makes that comparison under its own hypotheses and equates final CAL holders and reserve balances, not application hashes or revision metadata.

Local invalidation write set and replay. Invalidation is evaluated only while applying an authenticated block entry whose execution result is successful and applied; `INSTALL` does not trigger it. Before the follower overwrites a spend record, it reads the previous candidate and verifies the immutable approval and the exact principal (final or certified) or user-fee input instance. Existing observation, installed certificate material, an outbox entry, settlement, pending obligations, compensation, recovery, or a fee effect is treated as an accounting contradiction rather than as releasable state.

The additional invalidation write set contains only the affected local resource slices and the approval’s local progress. For each debit it subtracts Cap – Applied from Reserved, adds the same amount to Available, sets the corresponding cumulative Applied entry to Cap, and records Invalidated, the first triggering public fact, and its height. The immutable approval and unrelated input locks remain. Overlay visibility makes later conflicting inputs or transactions in the same block no-ops for restoration; the follower cursor suppresses block replay, and the cumulative Applied vector prevents a second credit after restart. The member-store schema is advanced so that an older binary cannot silently ignore the invalidated terminal state. These writes commit with the successful consumption record and follower cursor. Schema 8 rejects earlier member stores; experiments use fresh directories rather than a migration.

A fully certified source that is permanently rejected by the global Intent binding while spending disjoint inputs is outside this invalidation rule: its QC-backed responsibility, input locks, and signing reservation remain. The current relay may continue retrying such a source; terminating those retries is a separate engineering follow-up and cannot release the certified liability.

S2. ARCHIVED PER-RUN EVIDENCE: BUILD 721800C

This section records frozen evaluation build 721800c, retained as historical evidence; current-build continuation and representative P results are reported separately. Continuation and mixed-load trials share node binaries and Comet sources. The subsequent historical methods and archived tables retain their original build identities and are not throughput measurements of this build.

TABLE S1
ARCHIVED BUILD-721800C CONTINUATION; TIMES IN SECONDS FROM FIRST ACTUAL SEND.

Run	Last receipt	All public	Members closed	Attempts	Gaps
r1-fast	4.497998	5.228984	5.293857	100	3
r1-wait	64.626603	65.232083	65.284395	149	0
r2-fast	4.739972	5.253720	5.295722	100	2
r2-wait	64.475560	65.105699	65.154442	116	0
r3-fast	4.556805	5.114055	5.129723	100	4
r3-wait	64.460699	65.075768	65.124437	142	0

These costs come from offline successful-execution fixtures. Representation work includes sequential adaptation, assembly, Finalize, synchronous application Commit, and materialization; common compensation decisions are

TABLE S2

ORDINARY PAYMENTS PER MIXED-LOAD RUN; LATENCIES IN MS, SPAN AND DRAIN IN S. FINAL COLUMNS COUNT OUTSTANDING-LIMIT AND FAST-CONCURRENCY-LIMIT HITS.

Run	Receipt P50	Receipt P95	Dispatch P95	Span	Drain	Total cap	Fast cap
r1-C	53.435	136.221	201.339	70.191	1.029	1	0
r1-N	55.259	152.766	255.396	70.245	1.015	16	0
r1-P	55.102	154.780	895.879	70.886	1.356	63	935
r1-R	53.034	139.894	0.239	69.990	0.815	0	0
r2-C	54.011	136.490	700.329	70.690	0.906	68	760
r2-N	53.653	143.869	110.389	70.416	0.820	18	0
r2-P	52.892	134.963	0.242	69.990	1.066	0	0
r2-R	53.832	144.903	0.218	69.990	1.125	0	0
r3-C	55.682	159.437	405.296	70.395	1.046	65	2
r3-N	55.132	155.992	0.242	69.977	1.200	5	0
r3-P	54.996	144.206	586.541	70.576	1.155	81	549
r3-R	52.706	126.355	0.186	69.984	0.890	0	0

TABLE S3

HISTORICAL REPRESENTATION PER PROCESS; MAINTENANCE AND COMMON DECISION EXECUTION/COMMIT IN MS, NET ADDITIONAL KV IN BYTES.

Process	Payments	Compensated	Representation	Net extra KV	Common decision
1	32	1	43.322	292136	8.926
2	32	1	43.664	292200	8.917
3	32	1	42.035	292162	9.013
1	32	8	97.183	329118	11.919
2	32	8	94.461	329126	11.984
3	32	8	93.591	329059	12.029
1	256	1	64.826	2258078	9.052
2	256	1	66.931	2258044	9.989
3	256	1	65.323	2258085	8.993
1	256	8	214.398	2294947	20.119
2	256	8	213.902	2294910	21.027
3	256	8	214.781	2294980	20.967

reported separately. Network and consensus waits, public-block saving, and commit-signature construction are outside this timer. KV is the logical increase in encoded records, not the physical disk-space difference between two deployments.

Subsequent reclamation regression. Build `bf71c4d` adds the superseded-partial-approval transition. Directed tests cover exact ordinary and fee-input conflicts, duplicate processing, restart, refusal to re-sign, and the disjoint-input Intent counterexample. A separate fresh-start nine-service run with synchronous member storage completes 1,000 user-funded payments offered at 100 payments/s (9.990-s send span, 10.879-s total), and a 100-hop fast chain reaches its last certified receipt in 4.422 s and closes in 5.014 s. Both runs have matching committee state and empty outboxes. These correctness regressions do not replace the performance measurements of `721800c`.

S3. BOUNDARY AND NETWORK VALIDATION: BUILD 9879F64

Production snapshot `9879f64` retains the local reclamation patch `bf71c4d`. The new fixtures and network driver additions are fingerprinted independently; the measurements of `721800c` remain unchanged. The artifact directory `major-revision-2026-10-03` contains source manifests, raw logs, parsed ledgers and review closure.

Finite-capacity and Intent boundaries. Deterministic regressions obtain actual member approvals, execute blocks through `FinalizeBlock/Commit`, and authenticate their results with retained empty successor blocks before member following. The harness fixes ordering and block time.

TABLE S4
INTENT-CONFLICT LEDGER AFTER COMPENSATION UNDER A’S GRANT (CAL).

	Start	Round 1	Round 2
A’s reserve	300	200	100
Public Spent / Reserved	0/0	100/0	200/0
Reserved per source signer	0	100	200
Available per source signer	200	100	0
Available at fourth member	200	200	200

With $B_g = 300$ and one Worker per member, four independent 100-CAL requests each obtain two approvals, exhausting all four members’ 200-CAL capacities without a QC. A 150-CAL external reserve increase adds 100 capacity per member without deleting locks; the same requests then complete, leaving Available= 300 and Reserved= 0 per member. An all-honest same-input 2/2 split remains blocked after the same increase.

Two cross-organization Intent-conflict rounds run under both user-funded and authorized organization-funded fees. Organization A’s reserve falls from 300 to 200 and then 100 CAL; public Spent reaches 200 and Reserved returns to zero. The same three source signers each retain 200 locally, while the fourth retains its unused capacity; new requests cannot obtain a quorum from this slice. Authenticated failure results do not release the rejected approvals. Their original inputs remain publicly unspent and locally locked. User-funded runs additionally retain two complete 10,000-FUEL inputs, distinct from their 1,000-FUEL escrow maxima.

The first descendant funds the second A-side request, and the final descendant is successfully spent through B. Each B-side winner uses a different initial input, and the harness completes compensation before the next round. Independent sums check Open gaps, certificate coverage, grant usage, and CAL/FUEL conservation at every committed prefix. Repeated compensation produces no new changes, effects, or fee outputs. These results demonstrate finite reserve loss and conservative certification stoppage, not a network attack rate.

Fixed-signer capacity bound. Table S4 separates public reserve accounting from local signing capacity. With the same three source signers, one Worker per member, initially unused capacity, no top-up or release, and reservation $a > 0$ per source, this pattern admits at most

$$n_{\text{fixed}} = \left\lfloor \frac{\lfloor 2B_g/3 \rfloor}{a} \right\rfloor$$

rounds. Each round retains a at every honest source signer; when all of that source’s value is consumed and compensated, it also adds a to net reserve loss. Here $B_g = 300$, $a = 100$, and two rounds exhaust the three signers while 100 CAL remains in reserve. This is a bound for the specified signer pattern, not a maximum over arbitrary signer sets or schedules. Repetition also needs fresh paired Intents, independent winner inputs, usable descendant routes, and sufficient fees. A funded top-up adds the share difference without clearing prior reservations. No CAL–FUEL exchange value is inferred from this trace.

State-preserving restart and original-history replay. With real threshold adaptation, regressions cover both compensation–representation–repayment and compensation–repayment–representation. A replay replica stops at height 2, reopens its retained persistent ledger, and catches up using `Original` material from a `BlockStore` that already contains revised bodies. Within each fixed command history, application roots and ledger rows match the reference at every replayed height. This exercises local persistent restart and original-history access, not network downloading or equality of application roots across different command orders.

Fees and locked funds. Each successful normal payment in this trace spends 94 FUEL, and a compensated consumer spends 99. The two rounds plus the final descendant spend 480 FUEL: 430 rewards and 50 burned, out of escrow maxima totaling 5,000; 4,520 is returned. The user-funded trace separately retains two 10,000-FUEL inputs of the unexecuted sources. The organization-funded trace debits B’s authorized fee account by 480, with A’s public FUEL balance unchanged. A’s local fee reservations still remain. CAL loss, fee expenditure, and fee-input locks are distinct quantities.

Public entry and atomicity coverage. The `wire4` dispatcher admits `DirectSubmission`, reserve increases, clock ticks, and restricted repair commands. `DirectSubmission` reconstructs and verifies the current-organization QC,

owner authorization and direct-input certificates before common spend-state checks. Legacy ordinary transfers require CommitteeRoute and are excluded by the wire4 dispatcher. Organization-funded fees require a grant whose Policy Subject matches the payer; unapproved sponsorship is rejected at member and public paths. The zero/two-vote construction probes fail during canonical encoding; existing public-verification negative tests separately cover malformed or mismatched certificates. Compensation and representation cannot provide an alternative arbitrary owner-spend entry.

The synchronous approval boundary test interrupts immediately after the real database commit and before control returns to the signer. Reopening the same store preserves the reservation and input lock; the same request can be signed without another debit. Existing tests reject conflicting requests after restart, roll back cursor and state together on failed follower application, and apply restoration once across restart. These checks retain committed state rather than simulate its loss.

A. Waiting-Mode Timestamp Audit

Inter-hop observation cost. The driver polls every 5 ms and the block follower every 25 ms; 250 ms is the commit setting, not either polling interval. Matching the original 600 payment records to all four replicas' application-commit records identifies 99 inter-hop waits per waiting run. Receipt-to-public-observation intervals total 56.851, 58.921, and 57.031 s. For the same hops, earliest-replica-commit-to-driver-observation intervals total 29.542, 29.681, and 29.347 s, with per-hop P50 approximately 297 ms. The latter interval spans successor-header authentication, fetching and verification, local recording, observation, and replica differences; missing wallet-internal timestamps prevent further separation. Observation-to-next-build handoffs total only 0.273, 0.314, and 0.315 ms per run.

The 14.15-fold ratio consequently compares the two measured end-to-end modes in this configuration. It is neither a pure consensus speedup nor a polling-only penalty, and these intervals do not establish a counterfactual event-driven result.

B. Archival Byte-Export Example

The runnable `TestHistoricalExportConsumer` exports prefix, coordinates, full BlockID, exact Revision and Task, original and authorized bodies, and historical funding with current obligation state. It checks Task equality, owner authorization, QC, input openings, and reconstructed parts. The extended test compares original-body/index, plain cached funding rows, and C3 at $H = 4$ and after real repayment at $H = 5$; all return the same economic answer. Cached rows retain immutable fields and obtain status from the current read view. Both original and authorized bodies pass signature verification against the same saved original commit and validator set. A negative control retains the changed body and header but uses initial part openings: the part-set header differs and the commit check rejects the complete BlockID. Thus matching the header alone is insufficient for this consumer.

After repayment, authorized bytes, cached rows, and the permanent decision remain unchanged; current status changes from Repaired to Recovered. This local functional comparison does not time the cached view or establish external adoption. The main paper's read timings compare the original index and C3 layouts, not this third view. Authorization in every case comes from the executed prefix; the saved commit is an identity check, not independent permission to replace bytes.

C. Chameleon-Hash Instantiation

Parameters. The hash is an RSA chameleon hash with a 2048-bit modulus N and public exponent $e = 65537$. For context c and message m , the digest is $CH = H(c, m) \cdot r^e \bmod N$ with randomness r in $\mathbb{Z}_N^*$. Given the digest of (c, m, r) and a new message m' , the adapted randomness $r' = r (H(c, m) / H(c, m'))^d \bmod N$ satisfies $H(c, m') r'^e = H(c, m) r^e$, where $d = e^{-1}$ modulo the group order. Because $\gcd(e, \varphi(N)) = 1$ holds for correctly generated keys, $r \mapsto r^e$ is a permutation of $\mathbb{Z}_N^*$. Each message and digest then determine a unique valid opening.

Full-domain hash. H reads a SHAKE256 stream over the prefix `UTXO_CH_RSA_FDH_V3`, the 256-byte modulus, `u64be(|c|)`, the context c , `u64be(|m|)`, and the message m . It takes consecutive 256-byte blocks from the stream and returns the first value that is nonzero, below N , and a unit modulo N . The length prefixes make the encoding injective across contexts and messages. The contexts of block parts depend on the height and the part index and not on the revision counter.

Require: Batch B , state S , execution height

- 1: $b \leftarrow \text{BatchID}(B)$; if a successful Task b exists, return no new effect
- 2: Load Canonical; require an older target height and the stated base and previous digest
- 3: Check item order and unique outputs and slots
- 4: Require, for each item, a decision $\mathcal{D}_x$ at the stated location and a canonical slot holding `OriginalFunding` for x
- 5: Reconstruct the permitted combined funding replacements and verify their openings
- 6: Require the canonical candidate-body digest to equal $B.\text{Next}$
- 7: Verify non-target bytes, lengths, and that block hash and part-set header equal the stored original identity
- 8: $O \leftarrow \text{Overlay}(S)$; remove the pending entries and record per-output references
- 9: Add one logical revision, Task, and queue entry to O
- 10: Return the transition and a result without economic effect

Fig. S1. Atomic representation-batch execution. Failure rejects the whole batch, and only a successful transition commits. Batch identity covers the complete canonical command, including its proof data.

Threshold trapdoor. A trusted dealer uses `CIRCL GenerateKey` to generate two distinct safe primes and check the inverse of e modulo $\varphi(N)$, then uses `Deal` to share $\tilde{d} = e^{-1} \bmod \mu$, where $\mu = (p-1)(q-1)/4$, into four shares with Cloudflare CIRCL v1.6.5 `tss/rsa (Deal(4, 3, true))`, and the experiments never persist the complete private key. Each share holder signs the public ratio $H(c, m)/H(c, m')$ as a raw group element, not through a padded RSA signing API. `CombineSignShares` recombines the shares and the combiner tries at most four three-member groups until the final opening verifies against the public relation. The construction follows the threshold RSA approach of Shoup [1]. The raw-group interface is checked by its final public relation; the library reference alone is not a proof of that interface or of ledger authorization.

Raw-group correctness. Write $u = H(c, m)/H(c, m')$ and $\Delta = 4! = 24$. Interpolation of honest shares produces W with $W^e = u^{4\Delta^2}$. Since $\gcd(4\Delta^2, e) = \gcd(2304, 65537) = 1$, choose integers α, β satisfying $\alpha 4\Delta^2 + \beta e = 1$. The combiner obtains $z = W^{\alpha u^\beta}$ with $z^e = u$, hence $r' = rz$ preserves the commitment. The shared exponent $\tilde{d}$ modulo μ is distinct from the full-group exponent d used above to express the unique root. This explains honest-combination correctness for the unit-group ratio; it is not a claim of unforgeability for publicly combinable raw ratios.

Assumptions and scope. Next’s public-key constructor checks only that the modulus is a 2048-bit odd number and that $e = 65537$. It does not prove that the modulus is well formed, so the argument needs the dealer to generate and split the key correctly. The ratio is public and algebraically combinable, so the chameleon hash does not by itself establish that one authorization was issued for one replacement. An adaptation takes effect only through a committed compensation decision and an exact Task. These are checked independently of the hash. The original body is bound by the original parts at revision 0 with opening 1 together with the complete-identity checks described in the historical read interface of the main paper. This binding does not depend on secrecy of the trapdoor. Consensus safety still rests on the BFT, authentication, and state guards of the base protocol.

D. Representation Construction and Batch Checks

A constructor reads the latest committed logical representation, independently of physical installation, and selects decided outputs whose slots still hold `OriginalFunding`; the decision enforced affordability when it committed, so the constructor simulates no economic step. For every item it derives the reserve-debit identity from the network and the consumed output and replaces the slot’s `OriginalFunding` by the reserve reference with its opening, patching slots of one transaction into the same copy. Serialization lengths and all non-target bytes, including earlier authorized replacements, stay unchanged. Input adaptation runs per slot, and each block part that the combined body touches is adapted once while unchanged parts keep their commitments and proofs.

Share endpoints bind to fixed committee identities, and each holder derives the requested replacements from its own committed decisions and refuses a slot without one. The collector tests bounded three-member candidates by actual cryptographic combination and accepts a candidate only when every affected part verifies, so malformed or invalid responses eliminate candidates and leave three valid contributions from other members usable. A successful combination cancels outstanding requests. Openings give commitment compatibility, and the committed decision gives the authorization.

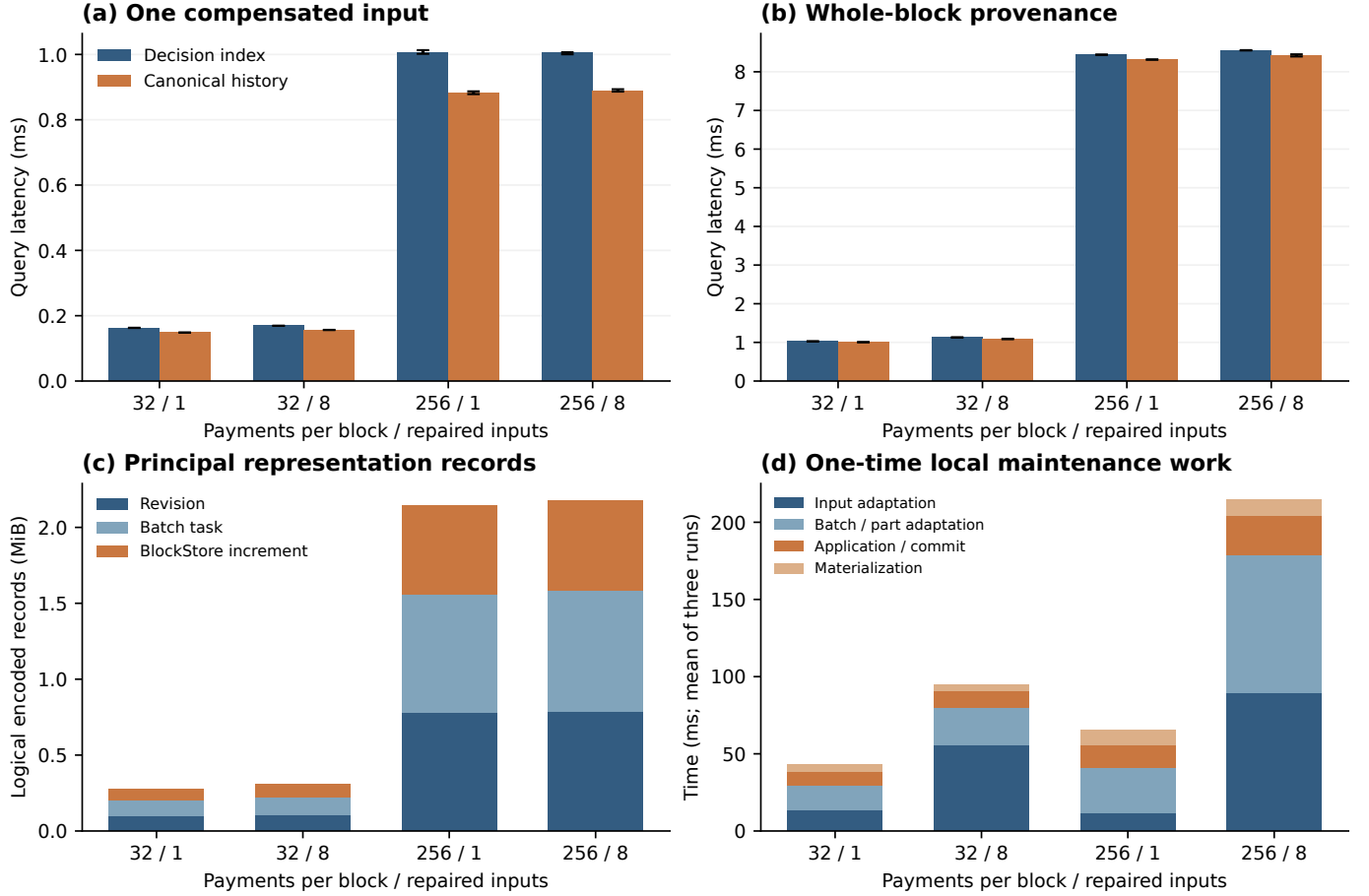


Fig. S2. Reader tradeoffs. (a,b) Medians and min–max ranges of three run-level P50s. (c) Mean major logical-record bytes. (d) Mean sequential stage costs, not the median total work in the historical-reader table in the main paper.

E. Additional Historical Reader Costs

From already decoded (256,8) run-1 objects, separate authorization rechecks cost about 26.3 ms per block on both paths, input-CH checks 10.3 ms, and complete BlockID reconstruction 0.67 ms. These independent rechecks are excluded from trusted-reader timings.

F. Injected-Latency Configuration and Per-Run Results

The fixed four-validator configuration runs one organization’s four members and one gateway. Each TCP direction uses a bounded FIFO of at most 64 chunks of 16 KiB, released at arrival plus the configured delay. All committee peer addresses use the proxies; duplicate-peer connection suppression can leave a listening proxy idle when that validator retains outgoing connections. Wallet–gateway traffic, public HTTP, wallet delivery and storage are not delayed. All twelve final runs use a file-descriptor limit of 16,384. The matrix was rerun after a pilot inherited a 256-descriptor limit; those tool-contaminated pilot results are retained separately and not pooled. Nodes stop before offline database audits.

The two chain durations in each run are measured from their first send to last certified receipt. Their median is first taken within the run, then across its three replications. The fault-window QC cohort starts at least one second after suspension and receives before resumption. All member-fault cohort QCs contain exactly members 0, 1 and 2. Offline verification checks all 91, 91 and 95 committed BlockIDs in the committee-fault runs. Fixed-set proposer priorities show five round-0 opportunities for the paused validator in each run; each commits in round 1 without its signature. Windows use the latest valid precommit timestamp rather than the previous height’s header time.

TABLE S5

PER-RUN RESULTS OF THE INJECTED-LATENCY STUDY (9879F64). PUBLIC AND MEMBER P95 IN S; CHAIN TIME IS THE MEDIAN AND MAXIMUM OVER THE RUN’S TWO 10-HOP CHAINS, IN MS.

Run	Receipt P50/P95 (ms)	Public P95	Member P95	Chain med/max	In-window public
H-1	42.88/87.52	1.277	1.361	577/619	–
H-2	41.56/83.20	1.288	1.375	599/619	–
H-3	40.68/78.23	1.263	1.344	515/542	–
L-1	81.91/118.22	1.315	1.410	946/980	–
L-2	83.73/123.52	1.360	1.468	1001/1029	–
L-3	78.05/111.80	1.307	1.394	815/835	–
L-M-1	77.65/112.85	1.281	29.002	844/846	599
L-M-2	78.31/115.68	1.327	29.026	842/869	601
L-M-3	77.71/108.46	1.304	28.473	881/908	603
L-C-1	78.18/110.94	12.627	12.744	906/965	510
L-C-2	77.33/109.32	10.018	10.042	916/992	400
L-C-3	77.41/108.44	9.516	9.618	873/901	421

G. Scheduled-to-Receipt Timing in the Archived Mixed Workload

For each of the 84,000 ordinary payments, we compute the recorded receipt timestamp minus its scheduled timestamp before taking quantiles. Using the archived nearest-rank convention and then the median of three run-level P95s gives 304.253, 140.007, 475.236 and 666.915 ms for N/R/C/P. Actual-send-to-receipt P95s are 152.766, 139.894, 136.491 and 144.205 ms. The former includes pre-send backpressure; neither is formed by adding separate P95s. Some third-run wall-clock records put sending before the scheduled timestamp (minimum -13.104 ms); raw differences and counts are retained, not clipped. These archived timestamps do not distinguish timer wake-up error from clock adjustment, and are not used to claim submillisecond dispatch accuracy.

S4. CURRENT-BUILD EVIDENCE AND IMPLEMENTATION CORRESPONDENCE

The production baseline is `f856c7b`. Six chain runs and three representative P runs share node binaries and Comet sources. Additional `payctl` following checkpoints are enabled in every chain run. The `final-evidence-2026-10-04` artifact retains source and binary fingerprints, payment records, audits, analysis scripts, and regression logs. Tables below are generated from their CSV/JSON records.

TABLE S6

SIX CHAIN RUNS. TIMES START AT THE FIRST ACTUAL SEND, IN SECONDS. PUBLIC DENOTES OBSERVATION OF PAYMENT 100; CLOSURE DENOTES ALL-PAYMENT MEMBER OBSERVATION.

Run	Receipt chain	Public	Closure	Attempts	Gaps
r1-fast	4.726	5.451	5.495	100	2
r1-wait	64.479	65.054	65.115	140	0
r2-fast	4.700	5.480	5.535	100	4
r2-wait	64.670	65.280	65.300	138	0
r3-fast	4.707	5.477	5.526	100	1
r3-wait	64.387	64.983	65.030	136	0

Stage definitions. Ordinary intervals use 100 payments per run and inter-hop intervals use 99 transitions. We calculate P50/P95 within each run, then take the median of the corresponding three quantiles. Shared wallet/height checkpoints are expanded by payment, not treated as independent height samples. Intervals use same-host Unix-nanosecond timestamps; whole-chain duration uses a monotonic clock. The verification checkpoint follows successful `VerifyBlock` at `PrepareBlock` entry. The post-transaction callback follows `Update` return, not a synchronous flush. Negative differences retain execution overlap. Successful-fetch time is contained in `commit-to-complete-fetch`;

neither overlapping intervals nor quantiles are added, and a successful return does not identify the earliest evidence availability.

TABLE S7
CLIENT STAGES: MEDIANS OF THREE WITHIN-RUN QUANTILES, IN MILLISECONDS.

Interval	Fast P50	Fast P95	Wait P50	Wait P95
Send to application commit	573.703	814.807	363.470	381.553
Commit to complete fetch	395.096	440.920	285.884	314.509
Fetch to verification checkpoint	0.114	0.399	0.084	0.236
Checkpoint to post-transaction callback	0.940	27.496	0.128	1.343
Callback to driver observation	2.669	4.649	3.099	4.713
Successful fetch call (overlapping)	9.228	26.355	0.557	7.593
Receipt to next send	0.269	15.228	585.442	625.777
Observation to next build	-905.118	-630.176	0.002	0.006
Next build to send	0.265	15.224	0.237	11.539

TABLE S8
P DRIVER BACKPRESSURE. LIMIT HITS COUNT DRIVER EVENTS, NOT REJECTED PAYMENTS. ALL PAYMENTS CLOSE.

Run	Peak outstanding	Total-limit hits	Fast-limit hits	Scheduled receipt P95 (ms)
r1-P	256	7	0	189.591
r2-P	256	29	0	326.828
r3-P	256	143	917	851.742

TABLE S9: Selected proof–implementation correspondences for production baseline `f856c7b`. The artifact index retains complete paths and test assertions.

Claims (main-text names)	Production entry and proof obligation	Representative regression evidence
Original-execution agreement	The Comet overlay and <code>matchesBlockID</code> compare both identity components. Original-part validation and candidate transfers preserve body-parts pairing and execution context through consensus and replay.	Stable-hash identity cases, rejected relabelled openings and uncertified revisions, and retained-original replay; Comet-specific evidence keeps its recorded build.
Authenticated, unique consumption; public authorization	<code>ApproveDirectBytes</code> commits locks, <code>Intent</code> , approval, and debits before signing. <code>CheckTx</code> , <code>ProcessProposal</code> , and <code>FinalizeBlock</code> reach <code>Engine.Check</code> and submission verification. Full-byte caching does not replace current-ledger checks in <code>EvaluateDirectPaymentAt</code> . The fixed wire4 cases in <code>Check/ExecuteAt</code> are <code>ReserveIncrease</code> , <code>ClockTick</code> , <code>RepairInput</code> , <code>RepairBatch</code> , <code>CompensationDecision</code> , and <code>DirectSubmission</code> ; <code>Execute</code> rejects legacy in direct mode.	<code>TestPublicAuthorizationAtABCIEntry</code> ; <code>TestDirectModeRejectsAlternatePaymentEnvelopes</code> ; distinct-voter, identical-byte-cache, current-ledger, durable-approval, and replay tests.
Superseded partial approval and Residual capacity	Authenticated successful, applied results reach <code>reclaimPartial</code> and <code>finishLocal</code> . Normal CAL restoration requires observed source settlement; invalidation separately requires the guarded same-configuration input conflict. Both restore only original residual debits, with cumulative progress and the follower cursor committed atomically.	Partial-conflict, multiple-input, failed-result, contradiction, late-effect, original-debit, and cumulative-restart tests.
Resubmission preserves authorization and Certificates bound liability, not execution	<code>recoverExposedSources</code> reconstructs retained approved material with the public QC and verifies it before queuing; it creates no approval or debit. <code>EvaluateDirectPaymentAt</code> preserves the global <code>Intent</code> binding established by successful execution.	Cross-organization and multi-depth source reconstruction, duplicate exposure, Subject separation, and the two-round <code>Intent</code> -compensation ledger.
Potential liabilities are funded	<code>PrepareDirectVector</code> , submission verification, and approval bind issuer, configuration, grant, and Worker availability. <code>GrantShare</code> , <code>EvaluateReserveIncrease</code> , and <code>ApplyReserveIncrease</code> use external unprotected funding and cumulative share differences without erasing existing debits. <code>Engine.NewEngine</code> aggregates CAL/FUEL grants by <code>AccountKey</code> and checks their total against its balance. Protected CAL includes all organization and CAL-grant accounts.	Capacity fragmentation, funded completion, persistent 2/2 input split, protected-source rejection, shared external funding, and cumulative rounding tests.
Compensation takes effect at most once and Source repayment	<code>ExecuteDecision</code> checks the recorded decision, <code>Open</code> status, public deadline, position, and amount before atomic compensation. <code>recoverOutput</code> repays the original issuer within successful source execution, bounds recovery by payment, and creates no second user output. Only an undecided target reaches <code>targetSlot</code> , which requires <code>OriginalFunding</code> matching its original <code>OutputID</code> , amount, and fixed consumer fields.	Repeated and conflicting decisions, late-source repayment, partial-output repair, four-hop orders, and split–merge orders.

Continued on next page

Table S9 (continued)

Claims (main-text names)	Production entry and proof obligation	Representative regression evidence
CAL conservation and CAL delay neutrality	Payment, compensation, repayment, and reserve-increase transitions apply through overlays and public commit. Failed transitions contribute no business changes. Terminal CAL comparison retains the theorem's initial-state, conflict-free source-closed payment-set, and successful payment/top-up conditions; it does not compare FUEL.	Monetary projections across interleavings and replay; current-build audits of 600 chain payments and 21,048 representative P payments.
Decision-gated representation authority and Representation is economically inert	InputTarget, batchBody, and ExecuteBatch derive decided slots, preserve non-target bytes, and check openings, version digests, and the original complete BlockID. Representation-only writes and unchanged subsequent economic reads are separate obligations; DecodePublicExecution adds no representation-induced principal effect. Both RepairInput and RepairBatch use decided-target guards. EvaluateDirectPaymentAt does not read historical Canonical. Existing decisions return before historical access; other revised slots leave an undecided target and its fixed fields unchanged.	Atomic two-input and cross-part batches, compensation before adaptation, and payment-repair monetary replay.
Identity preservation and installation independence; local reader contract	Canonical selects the logical revision in V_H . PrepareMaterialization supplies exact Task material; physical revision guards prevent rollback, and replay uses originals. Saved-commit compatibility does not itself authorize a revision.	Revision-order and interleaved-queue tests, original-history replay, reader equivalence, and same_consumer_views across real repayment at $H = 4, 5$.
Protected continuation; Economic closure does not wait for adaptation	Shared authorization guards compose with finality.VerifyBlock, PrepareBlock, and blockfollow.Commit. FreshRecv remains a history premise. Economic closure uses ExecuteDecision without adaptation access; business updates and follower progress commit together. wallet.ReceiveDirect checks registered configuration, network, owner, and c.VerifyOutput, and atomically records DirectCoin, rejecting inconsistent bodies for one OutputID. member.InstallDirectClassified verifies the full payment, rejects Invalidated in its transaction, treats Observed as a no-op, and records all input/fee consumption, install material, and outbox without another reservation. Receipt checks alone do not establish FreshRecv.	Durable approval/background installation, 297 early successors in six chains, and 96 replica-target P checks showing repayment before representation commitment or materialization.

Premises and evidence levels. The proofs cover finite traces under the stated model; production guards identify implementation responsibilities, and regressions exercise particular acceptance and rejection traces. Fixed four-member organizations and a four-validator committee, three-vote quorums, at most one static Byzantine participant per group, consistent configuration, and non-rollback of honest signing state remain model or deployment premises. Aggregate genesis backing and protected-CAL restrictions map to Engine.NewEngine, the fixed dispatcher, and the account guards above. Signature unforgeability, binding original commitments, and correctly generated dealer-based threshold RSA parameters are not established by functional tests; fixed-target opening convergence additionally requires unique valid openings. Historical reads trust one verified, executed prefix V_H with retained history, while progress retains its capacity, resource, and fair-delivery conditions. The new TestPublicAuthorizationAtABCIEntry sends already encoded, tampered authorization bytes directly through the three ABCI interfaces and checks rejection without business-state changes. Sender-side encoding failures are separate; receiver-side decoding, binding, and signature-verification rejections are distinct outcomes. This test does not identify the deepest rejection stage for each case; encoding or decoding rejection is not evidence that signature verification executed. Existing

cache regressions distinguish byte-identical reuse from current-ledger validation. The current eight-package regression passes, with client and test additions fingerprinted separately from the production baseline. Comet-interface, persistent-restart, network-evidence, and threshold tests retain their recorded versions and scope; they are not new remote-download experiments, exhaustive Byzantine-scheduling tests, or mechanized proofs.

A. Approval and Execution Conditions

TABLE S10: Member approval versus public execution. D: deterministic functions of the submitted bytes and static configuration. S: checks against two different states. L: delivery and scheduling assumptions. Theorem 3 compares only histories in which every payment and increase succeeds; the table does not claim that a QC-bearing payment always executes.

Check	Member approval (Approve DirectBytes)	Public execution (VerifyDirect Submission, EvaluateDirect PaymentAt)	Relation and claim
<i>D. Deterministic</i>			
D1 Encoding, owner authorization, initial funding openings, rule IDs, network, configuration, epoch	PrepareDirectVector; identity equals own organization	Same PrepareDirectVector, plus the current payment QC; organization looked up by configuration	Shared fixed-content checks. A member signs after local commit; public execution additionally verifies the assembled QC.
D2 Admission vector and caps (CAL cap includes change; escrow floor includes one compensation charge per certificate input)	PrepareDirectVector	PrepareDirectVector	Identical caps on both sides.
D3 Direct input certificates; value conservation	Issuer configuration, rules and output checked; one certificate-index entry per certified input; duplicate OutputIDs and unused entries rejected; $\sum \text{in} = \sum \text{out}$	The same bindings; $\sum \text{in} = \sum \text{out}$ at evaluation	Equivalent checks; the member also lies in a validated single-network configuration.
<i>S. State</i>			
S1 Input instances	Candidate/consumed locks; authenticated creation records for final principal and user-fee inputs, matching certificates for certified inputs. Locks are written before signing	Consumed flag; final creation record, or a direct certificate for a missing output	Quorum intersection: no two conflicting QCs. A QC excludes public consumption by a different fact. Local lag may cause rejection or permit a partial conflicting approval by a member outside the executed payment’s signer set; intersection still prevents a conflicting QC.

S2 Intent	Organization-local Intent $\rightarrow$ TxID	Global Intent $\rightarrow$ TxID, written on successful execution	Independent states; conflict is possible with valid QCs (the global Intent counterexample).
S3 Grants, capacity, fees	Worker-slice Available $\geq$ cap for every resource; fee inputs and grants checked in member state	CAL: registration within the grant (Reserved+Spent $\leq B_g$); non-CAL: updateUsage; FUEL account or fee-input UTXOs	CAL coverage excludes grant-capacity failure at first registration and reserve-principal shortfall when compensating an Open obligation. Other resource and fee checks remain separate execution conditions.
<i>L. Liveness</i>			
L1 Inclusion, delivery, resubmission	Original signer keeps the approval and follows the chain	Honest inclusion and fair delivery; no global Intent conflict for the source	No completion bound and no timeout unlocking; adaptation is not required for closure.

B. Recovery and Boundary Evidence

a) Automatic source resubmission (R).: On the frozen build, one run schedules 2,000 ordinary payments (100/s for 20s) plus three cross-organization parent-child pairs, 2,006 payments in total, with automatic resubmission enabled and adaptation not paused. For each pair, the driver obtains the parent’s three signatures through a collector that neither persists, installs nor submits it, and the driver never submits the parent itself; the client stays alive and observes, so this is withholding, not process-crash injection. The child spends the parent’s TXCer. After the child’s public execution exposes that certificate, the parent’s original signers rebuild and submit the parent. All three obligations end Fulfilled and none is compensated; all 2,006 payments close, committee state hashes agree, outboxes are empty, and member CAL, execution and byte-budget reservations return to zero. The three pairs are three recovery targets, not repetitions. Times are client observations; outbox creation times were not recorded.

TABLE S11

PAIR-LEVEL OBSERVATIONS IN R (MS). RECEIPT TIMES START AT EACH PAYMENT’S OWN FIRST SEND; THE LAST TWO COLUMNS START AT THE CHILD’S FIRST SEND AND ARE CLIENT OBSERVATIONS, NOT INTERNAL RECOVERY TIMES.

Pair	Parent receipt	Child receipt	Child public	Source Fulfilled
0	100.146	116.559	1391.849	2767.211
1	82.988	61.749	1806.036	2958.003
2	75.033	135.119	1286.147	2442.890

b) Rotating-signer bound.: A deterministic regression uses production approvals, public execution and compensation. One Byzantine member signs without applying its local limit; the three honest members co-sign in the pairs (0,1), (0,2), (1,2). Each round certifies a 100-CAL source that loses on global Intent; a valid child then consumes its output and the issuer reserve is debited. With $B_g = 300$ the reserve falls $300 \rightarrow 200 \rightarrow 100 \rightarrow 0$, honest reservations are (100,100,0), (200,100,100), (200,200,200), and the fourth request is refused by all three honest members. A repeated decision changes nothing and CAL/FUEL are conserved. The run attains $Q_g = B_g$, so the two-round stop in Table S4 is specific to a fixed signer set. It measures neither attacker profit nor cost: the source inputs stay locked and FUEL is spent.

c) Version bridge.: Between 721800c and 9879f64, five member production files changed (blocks.go, direct.go, outcome.go, partial_reclaim.go, source_recovery.go); the 721800c N/R/C/P and capital results therefore keep their original build and are not inherited. Between 9879f64 and f856c7b, the checked member, committee, rules, redaction, transport, wallet, protocol, finality, node entry points, dependencies and Comet production files are unchanged; only client and test files differ. The three redaction core files and the listed Comet overlay files have identical SHA-256 digests in all three builds. No older number is relabelled.

d) Retries in wait mode.: Wait mode uses 414 attempts for 300 payments (140, 138 and 136 per run). The client sleeps 50ms after every failed attempt, whatever the cause (transport, HTTP status or read error), and the

original logs do not classify failures. The 114 extra attempts therefore include 5.7 s of fixed sleep over the three runs, plus unmeasured service time. We report the $13.70\times$ ratio with retries included and derive no retry-free figure.

e) Manifest scope.: The manifest has 309 entries: 305 build-source files, two experiment runners, and `go.mod` and `go.sum`. Chain, P, and R node binaries, the client, and expanded Comet sources match exactly. The member wrapper executes `member-real`. It sets `GOMAXPROCS` to 16 and `GOGC` to 200.

f) Dispatch-slot interpretation.: The driver acquires a fast slot before serialized pacing and releases it after certified receipt. In P run 3, 63.31% of aggregate slot time precedes HTTP send; its P95 is 589.422 ms, compared with 589.418 ms of recorded pacing wait. At the 917 fast-limit checks, timestamp reconstruction gives medians of 59 acquired-but-unsent tasks and five sent tasks awaiting receipt. The reconstruction uses the receipt timestamp immediately before slot release. Thus a fast-limit hit does not identify a slow member request. Total unfinished-task backpressure remains in the results; no runtime trace was collected to attribute its underlying service delay to CPU scheduling, storage, or block following.

S5. EXPERIMENTAL METHODS FOR ARCHIVED BUILDS

Build lineage and deployment. Results retain their original build identifiers (Table S12). The *continuation and accounting baseline*, committed as `d99ec92`, implements the layered continuation and the accounting transitions; its compensation commits together with its representation. The artifact records per-binary SHA-256 fingerprints and the configuration of each run. The nine-service continuation, compensation and repayment, storage, and short-throughput series use this build. The initial fourteen-service paired runs use the same single-obligation rule before the batch-result tag was separated from the direct-result tag, and the cross-type codec regressions cover that separation. Members in the baseline storage, in-memory continuation, and short-throughput controls run with `GOGC=200` and `GOMAXPROCS=16`; this override is not applied to committee, gateway, or wallet processes.

The *decision-first build* (accounting domain `DIRECT_ACCOUNTING_V5_DECISION_FIRST`) implements re-submission from public certificates, the compensation decision, and the separate representation commands. It derives from the branch base `0e7bae5`, a documentation-only descendant of `d99ec92`. The nine functional runs use the initial implementation snapshot, whose source-package SHA-256 begins `2c91b0251024322c`. The later snapshot of this series, whose package SHA-256 begins `449b4d4072c99588`, rejects a decision whose position does not match committed state before reading any historical block and raises the bounded due-decision scan from 8 to 32 entries per round; the ordinary payment path is unchanged. The six ordinary-path runs, the full test suite, and the race tests use the later snapshot of this series. The audited commit `f185f73` additionally requires the certificates attached to a payment to correspond one-to-one to its certificate inputs, and it does not change the payments accepted in the measured workloads. Numbers from the baseline and decision-first builds are reported separately.

The archived operating-range runs record base `e3870e1` and implementation `a083ff6`. Original continuation records `f0c0a274`, capital records base `8e1fb40` and implementation `5807546`, and organization paths record base `bb65af69` and verified source `13be856`. The archived batch integration uses `f82f8cd`, and the structural repair counts were repeated on the baseline code. The source-failure checkpoint `c1ebf2d` uses the superseded late-source rule and does not support the current claims.

Mac trials share one M4 Max host with 16 CPU cores and 64 GiB RAM, and the structural microbenchmarks run on Windows amd64. Baseline memory chains and short throughput trials use in-memory service state and NoSync wallets. Compensation and repayment trials use disk-backed committee application state and BlockStore with in-memory members and gateway. Storage controls change only member commits between bbolt NoSync and synchronous transactions. Each run starts from fresh genesis. Six additional fourteen-service disk-backed chain runs use the baseline accounting rule and user-paid FUEL. Their three fast 100-hop runs take 668.960–688.080 ms, and the inter-hop confirmation runs take 114.496–118.742 s. They form a separate storage and deployment configuration from the nine-service results in the main paper.

Decision-first functional runs. The nine runs use the fourteen-service configuration with 450 initial outputs, a disk-backed committee, members in their default NoSync mode, flush and gossip settings of 10 ms, and explicitly authorized organization-funded FUEL; signatures and all amount and consumption checks remain enabled. Ordinary-payment stress wallets use the NoSync wallet option, and the two-hop fault demonstrations use the default wallet mode. In each run, an injected fault prevents the original publisher from submitting the parent payment or running INSTALL for it after its full QC exists, so the original signers hold their approvals but not the QC. The child spends the parent’s TXCer. In the resubmission treatment the driver never submits the original payment. The

TABLE S12
BUILDS AND THE SERIES THEY SUPPORT.

Build	Identifier	Series
Continuation and accounting baseline	d99ec92	100-hop chains, synchronous-member and storage controls, compensation and repayment matrix, paired interference runs, short throughput.
Decision-first	Base 0e7bae5; functional package 2c91b0251024322c; later series snapshot 449b4d4072c99588; audited f185f73	Nine functional runs, six ordinary-path runs, regression and race tests.
Archived evaluation	721800c	100-hop chains, the complete N/R/C/P matrix, and historical-reading controls (Section S2).
Partial-approval reclamation	bf71c4d	Conflict and restart regressions; 1,000-payment smoke and 100-hop continuation.
Archived	e3870e1/a083ff6, f0c0a274, 8e1fb40/5807546, bb65af69/13be856, f82f8cd, c1ebf2d	Operating range, original continuation, capital, organization paths, batch integration, source-failure checkpoint.

control treatment disables the automatic resubmission step with an experiment switch. Ordinary backup submission by members holding installed complete material remains enabled; INSTALL itself replicates that material within the organization. The paused treatment also disables resubmission and, while a marker file exists, makes the input and part adaptation HTTP endpoints of all four committee members return a paused status; consensus, decisions, and payment processing are not paused.

Times start at the first HTTP send of the child payment. The fast receipt ends when the wallet has verified the certificate and recorded the output. Public events are observed by polling the followers' status endpoints, so they include polling delay, and the difference between two public observations is not a protocol delay measured from the exact commit. The compensation time includes the configured 30-s obligation period, the advance of public time by committee ticks, and background scheduling. In the paused treatment the script submits the late source after the compensation and waits until the status endpoints of all four committee members show the obligation repaid and the representation neither committed nor materialized. It stores these four observations, removes the marker, and then waits for the representation to be committed and materialized at all four members. The status endpoint's stability flag compares the header hash, data hash, and transaction identity. The materialization step and the real-consensus integration tests check the complete block identity and part-set header, so the flag alone is not offered as the complete identity proof. The repayment and the resumption of adaptation have no timestamps of their own. The measurements contain no comparison with periodic scanning, vote collection, or the earlier coupled build under paused adaptation.

Each treatment has three rounds, and every functional run passes four checks: equal state hashes at the four committee members, a zero gap, closed payments, and drained outboxes. The resubmission and control treatments each carry three additional runs of 300 ordinary payments. Per-run values are in Table S13.

Decision-first ordinary-path runs. Six runs compare the build at the branch base with the final decision-first snapshot in the order AB, BA, AB. Each sends 300 payments at 100 payments/s with 64 concurrent fast requests and at most 256 outstanding, using NoSync stress wallets. The elapsed time includes the public confirmation of all 300 payments and member following; it is neither a single-payment time nor a maximum throughput. The send span is about 2.990s in every run. Summary values in the main text are medians of the per-run statistics, not quantiles pooled over payments. Ordinary final outputs carry no input certificates, so these runs do not measure the storage retained for payments whose inputs are not final. Per-run values are in Table S14.

Regression coverage. Compensation decisions are rejected when they are not yet due, when the network, slot, or amount does not match, when the reserve is insufficient, or when they conflict with the recorded decision. A repeated decision has no economic effect, including after repayment, where it returns no changes, effects, or fee outputs, and both orders, source first and decision first, are covered. Resubmission is tested for cross-organization

TABLE S13

PER-RUN RECORDS OF THE NINE DECISION-FIRST FUNCTIONAL RUNS. TIMES ARE CLIENT OBSERVATIONS FROM THE CHILD PAYMENT'S WALLET SEND; "PARENT FAST" IS THE ORIGINAL PAYMENT'S OWN RECEIPT. PUBLIC COLUMNS INCLUDE POLLING. A DASH MARKS A COLUMN THAT DOES NOT APPLY OR WHOSE TIME WAS NOT RECORDED. "AUTO. SOURCE" DOES NOT APPLY IN THE CONTROL AND PAUSED ROWS BECAUSE AUTOMATIC RESUBMISSION IS DISABLED THERE; THE DRIVER LATER SUBMITS THE SOURCE, WHICH EXECUTES AND REPAYS THE ISSUER (VERIFIED BY THE REPAYMENT AUDIT), BUT THAT INSTANT WAS NOT TIMESTAMPED. IN THE RESUBMISSION ROWS NO COMPENSATION OR MATERIALIZATION OCCURS.

Treatment	Run	Child fast (ms)	Parent fast (ms)	Child public (s)	Auto. source (s)	Compensation (s)	Materialized (s)
Resubmission	1	13.94	9.02	1.414	2.014	—	—
	2	13.94	8.94	1.990	2.565	—	—
	3	13.91	9.02	1.440	2.039	—	—
Control	1	11.96	8.97	1.364	—	35.839	36.994
	2	12.94	8.97	1.413	—	35.888	37.021
	3	12.01	9.00	1.388	—	35.864	37.019
Paused	1	12.00	7.98	1.362	—	35.837	42.748
	2	13.92	9.04	1.341	—	35.814	42.736
	3	13.92	9.03	1.841	—	35.891	42.798

TABLE S14

PER-RUN RECORDS OF THE SIX ORDINARY-PATH RUNS. RUN ORDERS ARE AB, BA, AB (A = BRANCH BASE, B = DECISION-FIRST). QUANTILES ARE OVER THE 300 PAYMENTS OF ONE RUN.

Build	Run	Elapsed (s)	Fast P50 (ms)	Fast P95 (ms)	Block P50 (ms)	Dispatch P95 (ms)	Send span (s)
Branch base	1	3.871	1.112	27.441	1151.0	0.338	2.990
	2	3.815	1.096	17.951	1101.1	0.295	2.990
	3	3.805	1.052	39.945	1076.0	0.437	2.990
Decision-first	1	4.000	1.071	30.530	1127.9	0.430	2.990
	2	3.845	1.073	22.084	1103.9	0.355	2.990
	3	3.791	1.045	33.744	1052.3	0.383	2.990

and multi-level sources, repeated exposure, and payments already observed or already queued, which are not rebuilt. Ordinary verification rejects mismatched material, and the original approval, the payment bytes, and the CAL quota are unchanged before and after resubmission. Representation requires a successful decision, remains possible after the source's repayment, and does not restore member progress or Worker slices a second time. For identity and replay, real threshold adaptation preserves the complete consensus identity, the successor body, and pre-signed successor references. Replay of the original history reproduces the application state at every height, and item-wise and differently grouped representation of one decided set give the same final body and part openings, with 1, 2, 4, and 8 inputs spanning two real parts. The full test suite and static checks pass at the later snapshot of this series on Windows and macOS, the race detector passes on the member, representation, and committee-runtime packages on macOS, and the full suite and static checks pass again at the audited commit.

Timing and aggregation. Certified-receipt latency generally runs from the first wallet HTTP send through recipient verification and local atomic recording. Capital and source-failure drivers include a small amount of pre-send construction, so their comparisons stay within the same driver family. A local NoSync transaction completion is not a synchronous durability receipt. Public and member completion timestamps record observation of authenticated execution, not the committee's internal commit. The sender's single monotonic clock measures cross-site ACK latency, including the return path, and a recipient can start its successor before the preceding sender receives that ACK. Offered targets, actual send rates, and completion rates including drain are retained separately. Captions distinguish transaction quantiles from statistics across runs, and within-run sample variability is not an estimate of cross-run uncertainty.

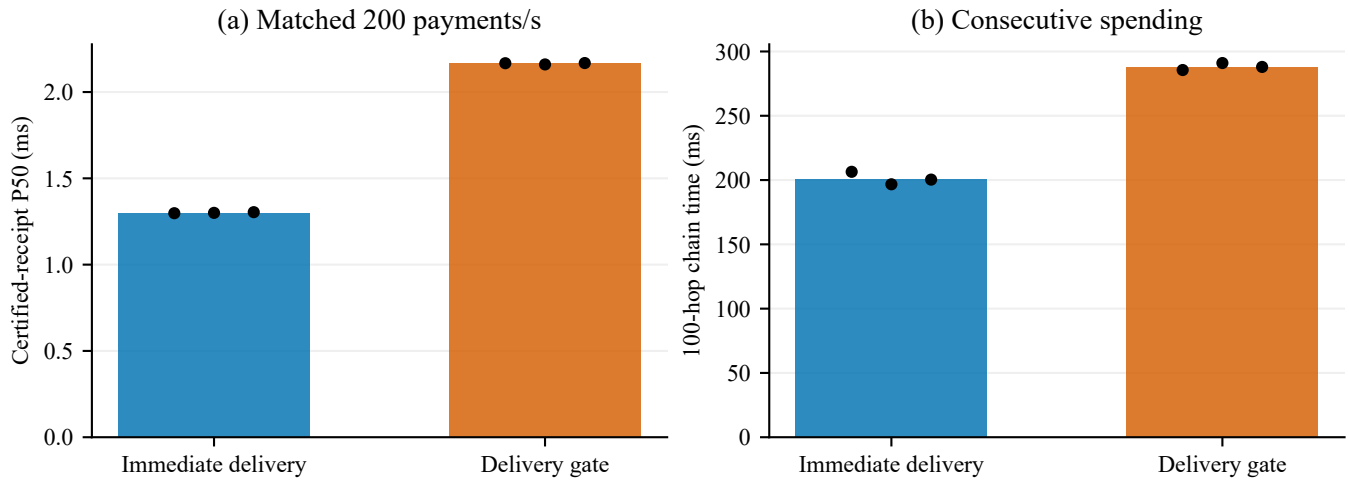


Fig. S3. Delivery-gate effect (ms). Left: run-level receipt P50 at 200 payments/s, 6,000 payments/run, three paired repetitions. Right: whole-chain time for three 100-hop chains per mode. Columns show the median of the three run values; dots show individual runs.

Delivery-gate runs. On the archived delivery-gate build, the gated variant waits for three members to INSTALL complete material or for authenticated public success, whichever occurs first, and collects no second signature quorum. At 200 payments/s for 30 s, three paired runs per mode each send and complete 6,000 payments. The median of run-level receipt P50s is 1.300 ms with immediate delivery and 2.167 ms with the gate. Separate 100-hop trials reduce median chain time from 287.983 to 200.356 ms (Fig. S3).

Baseline-build controls. The reserve trajectories of the three 5% runs of the compensation and repayment matrix (Fig. S4) are sampled at the driver’s release schedule, so the lowest sampled reserve is not a reserve-sizing result.

Three pairs of fourteen-service runs, ordered normal/withheld, withheld/normal, and normal/withheld, use the initial baseline build, before the batch-result tag separation, with committee and member databases on disk and member and wallet NoSync. Each sends 4,000 payments at 100 payments/s for 40 s. Alongside them, four additional two-hop demonstrations have sources that are delivered normally (normal run) or withheld, compensated after 30 s, and later repaid (withheld run). All 4,000 payments complete in all six runs. The final CAL held by each owner role (30,000 and 830,000) and both reserve balances (10^{12}) coincide across the six runs, and the withheld runs record 400 CAL paid and 400 CAL repaid, so the final allocation is consistent with delay neutrality. Receipt P50s for normal and withheld runs are 1.070/1.127, 1.089/1.108, and 1.083/1.097 ms, and P95s are 26.84/59.23, 29.49/58.44, and 27.41/55.15 ms; completion rates are 97.2–97.8 payments/s in all runs. Receipt P50 is 1.3–5.3% higher and P95 is 1.98–2.21 times the paired normal values. Concurrent compensation therefore affects the receipt tail in this configuration, and these pairs do not isolate the responsible stage.

Six runs of 1,000 payments at 100 payments/s on the nine-service configuration (committee on disk, wallets NoSync) differ only in the member store. With NoSync members, receipt P50 is 1.002–1.035 ms and P95 33.97–40.62 ms, with 93.11–93.93 payments/s completed including drain. With synchronous member commits, P50 is 43.19–51.85 ms and P95 80.98–96.56 ms, with 90.99–92.72 payments/s. All payments complete. Because wallets remain NoSync, this isolates the cost of synchronous member commits.

Representation microbenchmarks and batch integration. These structural measurements use the earlier coupled representation command, which also carried the economic step. Microbenchmarks on Windows amd64 with Go 1.27.1 place $k \in \{1, 2, 4, 8\}$ missing inputs in one historical block and part. Across five repeats, single commands require k commands and k part adaptations, whereas a batch uses one of each, and input adaptations remain k . At $k = 8$, total encoded command bytes fall from 127,336 to 2,752 (Fig. S5). The dispersed control separates batching from compact encoding: eight independent target blocks still require eight commands and eight part adaptations, while bytes fall from 30,008 to 5,440. At $k = 1$, both formats perform one part adaptation and encode 3,751 versus 680 bytes. Compact fields therefore explain part of the byte reduction, and shared block representation explains the reduced adaptation count. These are encoded command sizes, not total network traffic or total cryptographic work. They also exclude the 88-byte decision command that each decided output carries in the decision-first build.

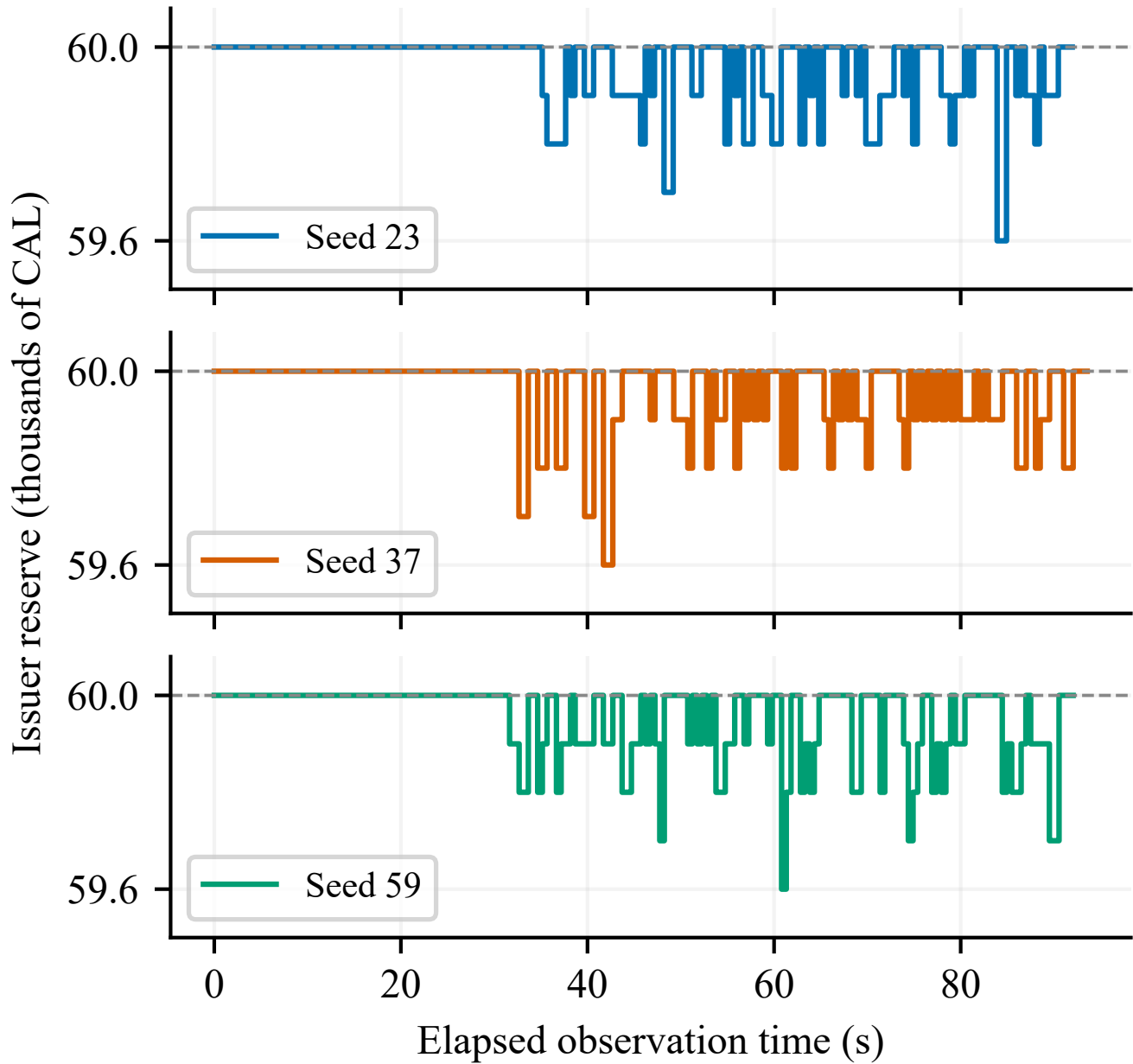


Fig. S4. Sampled reserve trajectories for the three 5% runs (60 compensated sources each). Repayment restores the initial 60,000 CAL. The vertical axis is restricted to 59,600–60,000 CAL to show temporary use. Samples reflect the driver’s release schedule.

An archived fourteen-service, disk-backed batch-integration run (`f82f8cd`) combines four actual missing sources into a single 1,568-byte batch whose BatchID agrees across all committee replicas. After late-parent handling, all 1,000 subsequent payments offered at 100 payments/s complete. Receipt P50/P95 are 1.048/26.531 ms, and total completion time is 11.120 s, including observation and shutdown. Gaps and outboxes drain, and committee state hashes agree. This batch carried economic steps and predates the separation of decisions from representation.

Externally funded authorization controller. The capital experiments sample member availability every 200 ms. Let ρ be the declared CAL authorization demand per second, counting both parent and child requests in each

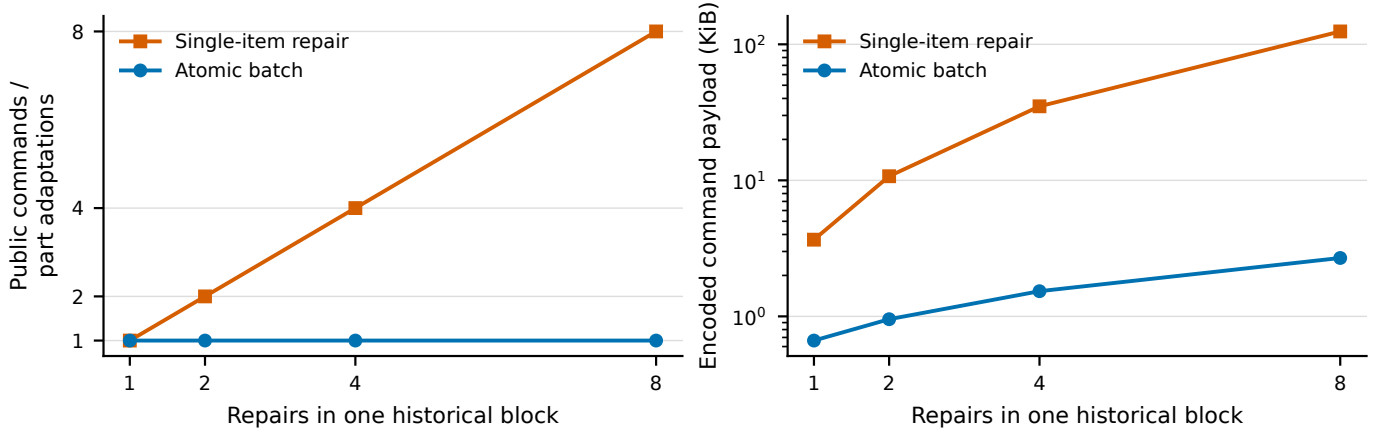


Fig. S5. Same-block, same-part representation costs of the earlier coupled command versus obligation count k . Panels show command/part-adaptation counts and total encoded command payloads in KiB (log scale). Five repeats per configuration give identical structural measurements. Input adaptations remain k .

TABLE S15

ACTUAL CAPITAL ADDITIONS. EACH CONDITION HAS ONE 300-s RUN, STARTS WITH 3,600 CAL, AND COMPLETES 12,000 PAYMENTS. BOTH COLUMNS ARE CAL; EVERY ADDITION DEBITS AN EXTERNAL FUNDED ACCOUNT.

Workload	Final reserve	Added capital
Constant: 20 units/s	14,400	10,800
Step: 10 to 30 units/s	21,600	18,000
Parent delayed by 3 s	23,300	19,700

workload unit, and let u count unique requests whose certification remains unfinished. The controller sets

$$\begin{aligned}
 b &= 200 + 100u, \\
 H_{\text{low}} &= \rho(0.8\text{ s}) + b, \\
 H_{\text{target}} &= 1.25H_{\text{low}}.
 \end{aligned}$$

Here the watermarks and burst allowance are in CAL. When the minimum observed member availability A_{min} falls below H_{low} , the proposed grant increment is

$$\Delta B = 100 \left\lceil \frac{1.5(H_{\text{target}} - A_{\text{min}})}{100} \right\rceil,$$

subject to the configured authorization cap and available external funding. At most one increment is in flight. Its public command debits the external source and credits the reserve and grant atomically, and following members add the resulting share difference without clearing existing approvals. The experiment supplies ρ in advance rather than estimating unknown production demand. Controller decisions are requests for actual funded transfers, not a mechanism for creating backing capital.

Capital workload accounting. Each condition has one 300-s run with one Worker per member and sufficient user FUEL. Starting from 3,600 CAL, the constant, rate-step, and delayed-parent conditions end at 14,400, 21,600, and 23,300 CAL after external transfers of 10,800, 18,000, and 19,700 CAL (Table S15). Each completes 6,000 two-hop units, or 12,000 payments, spending 1,128,000 user FUEL. Corresponding fixed-high controls complete, while the fixed-low condition closes 46 of 6,000 units, leaves 5,826 unstarted, and records 64 partially approved payments. These categories have different units and are not summed as one payment count. The per-run tables preserve the full seven-condition comparison without implying three repetitions per condition. The series ran on an earlier build in which no compensation occurred.

Identities, resource use, and retained history. The multi-identity configurations use shared clients, so 2,048 wallet identities do not represent 2,048 separate operating system processes. Comparing independent inputs with ten-hop

TABLE S16

COMPENSATION DECISION RECORD AND REPRESENTATION COMMANDS (C3). BOTH ARE PUBLIC COMMANDS COMMITTED ONCE; THEY DIFFER IN WHAT THE STORED HISTORY SHOWS.

	Decision record	Decision plus representation (C3)
Economic closure	Reserve debit, obligation state, and fee effects commit in one public decision, independent of adaptation; successors unchanged.	The same decision; representation commands add no economic effect.
Stored historical body	Original body unchanged; the funding source of a consumed input is read from the decision index.	Stored body names the reserve debit; transaction identity, owner authorization, and complete block identity (H_b, P_b) are unchanged and verified.
Inspecting a designated historical input	Consult the decision index together with the block.	Read the represented source under the original commitments and consult the permanent decision and current obligation state in the same V_H .
Readers	An honest local replica that has verified and executed prefix H .	The same local replica, reading a committed logical revision in V_H ; remote untrusted readers need separate authorization evidence.
Storage and replay	Original blocks plus decision records; replay runs the original commands, including decisions.	Additionally revised bodies, revisions, and tasks; replay runs the same original commands and the representation commands; same-block commands share part adaptation and installation advances to the latest committed revision.
Progress dependency	Public execution of the decision.	Three adaptation contributions and a current revision; economic closure does not depend on them.
Status	Implemented as the economic path; measured in the decision-first runs.	Implemented; functional runs and regressions above.

payments, the three-run medians are 12.67 versus 15.24 core-ms per payment and 59.32 versus 71.83 KiB of HTTP payload per payment. Memory reporting separates total node RSS and retained history from active obligations, pending work, and outboxes, so an increasing historical footprint is not reported as an increasing payment backlog. Per-run resource summaries and their archived source locations are included in the artifact index.

Endpoint-specific Lightning reference. The archived deployment uses LND v0.21.3-beta and Bitcoin Core 31.1 in regtest, with synchronous LND persistence and its 50-ms channel-commit setting. Each of three fresh deployments has two nodes and a direct channel. After 20 warm-up transfers, each run issues 100 serial, alternating 10,000-satoshi payments using prepared invoices, with receiver `SETTLED` as the completion event; the reported P50/P95 are 282.12/320.17 ms. A separate Next run sends 300 payments at 100 payments/s for 3 s using in-memory service state and NoSync wallet storage. Recipient-certified receipt has P50/P95 of 0.9625/1.2075 ms. Workload concurrency, durability, and completion semantics differ, so these records are deployment references rather than a normalized acceleration ratio.

A. Archived Operating-Range Series

The archived operating-range series complements the continuation results. One active organization uses sponsored fees, reused addresses, and independent final inputs. Five runs of 150,000 payments each all finish (Fig. S6). At a target of 2,000 payments/s, two runs achieve completion rates of 1,948.66–1,950.45 payments/s, including drain, with receipt P50s of 2.521–2.566 ms. Raising the target to 2,200 yields 2,103.63–2,106.29 completed payments/s. At 2,400, the achieved rate is 2,174.68, P50/P95 rise to 7.174/89.38 ms, and the 4,096-entry pending admission limit records 1,635 hits. The operating curve pairs achieved completion rates with their corresponding queue pressure.

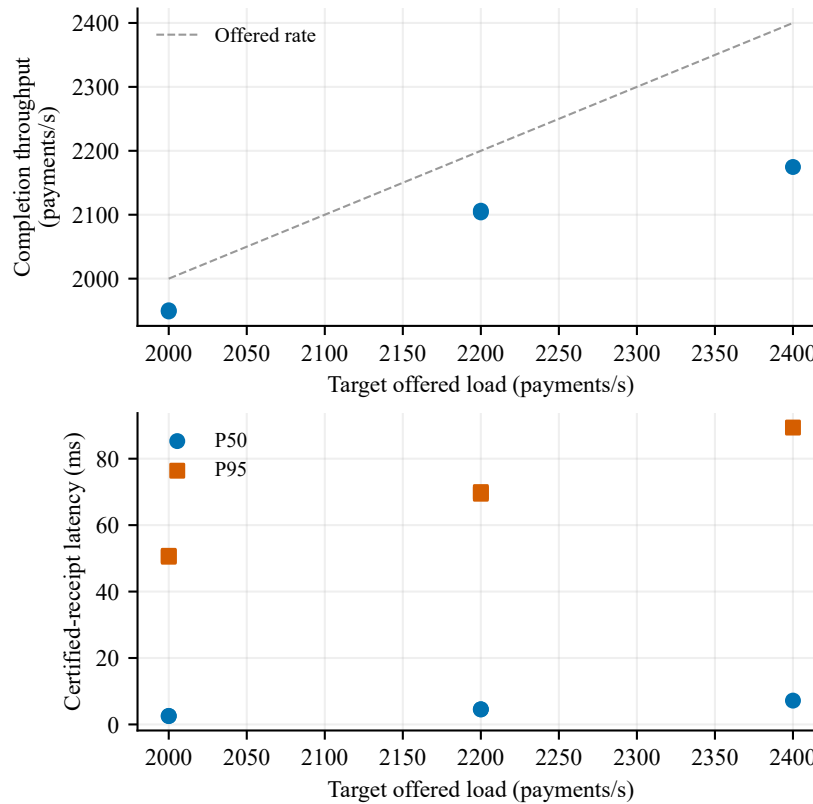


Fig. S6. Operating points versus target offered load. Top: completion throughput (payments/s), including drain. Bottom: receipt latency (ms). Each point is one 150,000-payment run: two at target 2,000, two at 2,200, and one at 2,400. P50/P95 summarize transactions within each run, not cross-run uncertainty. All services share one host.

S6. RESULTS FROM EARLIER BUILDS

These archived results retain their original builds, storage modes, and measurement endpoints. The main paper reports continuous-payment and representative P results on `f856c7b`; historical-reader results retain `721800c`, and the injected-latency study retains `9879f64`. Results from different builds and workloads are not combined into a capacity estimate for one configuration.

A. Continuous Spending Without Per-Hop Settlement

We compare certified continuation with waiting for public execution between hops. On the baseline build, a 100-hop chain reaches its last certified receipt in a median 158.846 ms, compared with 65.204 s when each hop waits for public confirmation. Three wallets transfer 100 CAL in rotation, constructing each payment online from its predecessor’s output and paying fees from their own final FUEL. Three repetitions per mode give six runs and 600 payments on the nine-service in-memory configuration. Both modes end at the last certified receipt, and the waiting mode adds a public confirmation *between* hops. With committee commit, flush, and gossip settings of 250, 10, and 10 ms, public execution and its observation cost about 0.65 s per hop. Certified continuation removes that wait from the recipient’s path.

Figure S7 shows the six whole-chain durations. The fast runs take 158.492–159.905 ms. All 297 fast-mode successor requests consume the preceding certified output and are sent after its certified receipt but before its public completion is observed. A one-input, one-output TXCer is 779 bytes, and requests are 1,979 bytes at the first hop and 2,766 bytes thereafter. For this transaction shape, the direct evidence stays the same size across all 100 hops, because each request carries the certificates of its direct inputs rather than the ancestor chain.

To test whether the advantage depends on in-memory member storage, three paired 100-hop trials on a nine-service disk-backed configuration with NoSync wallets and user-paid FUEL differ only in member storage (NoSync or synchronous; order N/S, S/N, N/S), and each is followed by a synchronous waiting chain. Median durations are

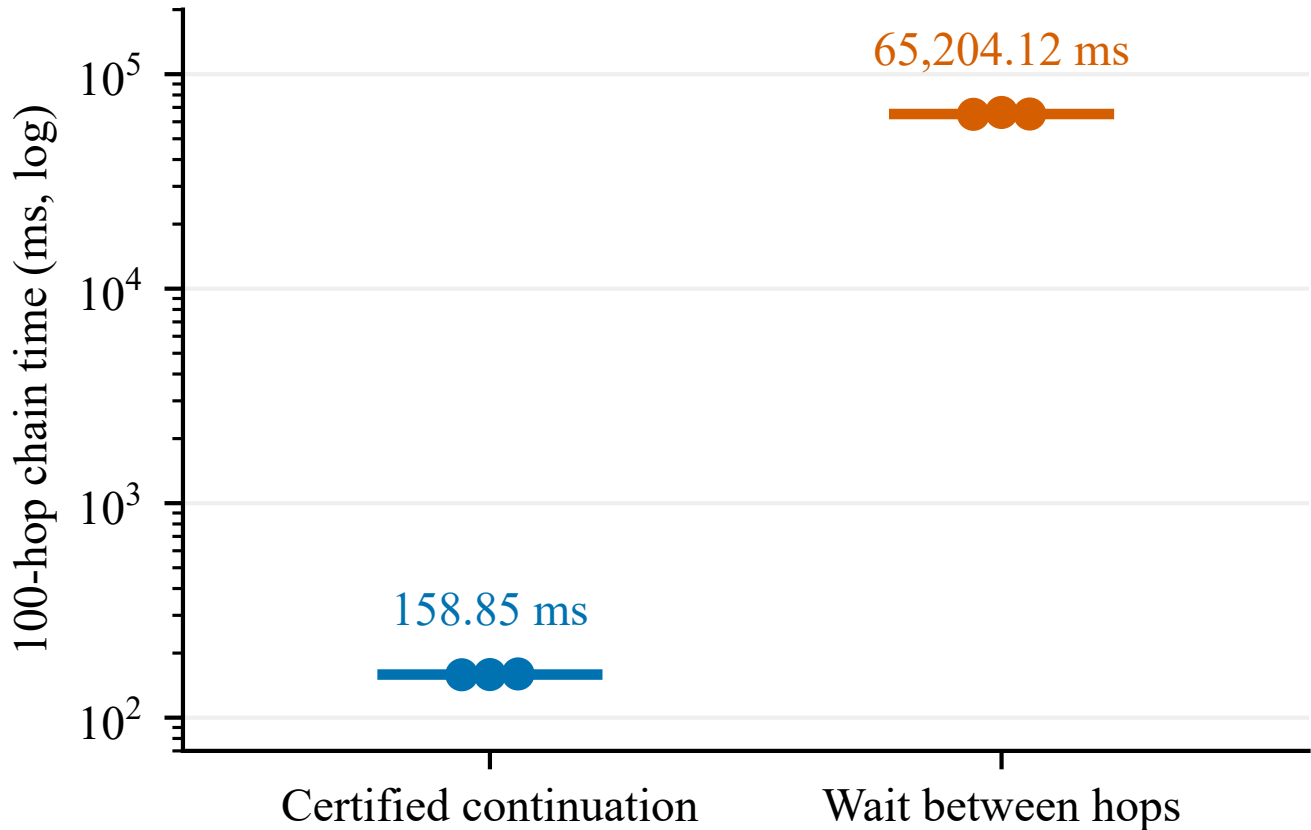


Fig. S7. Time through the last certified receipt of a 100-hop chain on the baseline build. Dots are three independent runs per mode; horizontal bars mark their medians. The logarithmic time axis includes the same endpoint in both modes. Only the waiting baseline requires public confirmation between successive payments.

681.127 ms for NoSync continuation, 4.471 s for synchronous continuation, and 65.013 s for synchronous waiting. Certified continuation therefore remains 14.5 times faster than waiting when member commits are synchronous. All 900 payments complete with agreeing committee states.

On an archived delivery-gate build, a gate after certification waits for three members to INSTALL complete material or for authenticated public success. The gate collects no second signature quorum, and delivering immediately instead shortens continuation. At 200 payments/s for 30 s, three paired runs per mode each complete 6,000 payments, and the median of run-level receipt P50s falls from 2.167 ms with the gate to 1.300 ms (40.0%). Separate 100-hop trials fall from 287.983 to 200.356 ms (30.43%). Authorization is unchanged. Fig. S3 shows the run-level results; Section S5 gives the methods.

B. Funded Closure and Repayment

This experiment checks that compensation leaves successor payments in place and that each compensation is repaid exactly once when its source later executes.

Compensation and repayment matrix (baseline build). Nine runs use seeds 23, 37, and 59 at withheld-parent rates of 0%, 1%, and 5%. Each run offers 20 two-hop units/s for 60 s (1,200 units, 2,400 payments) against a fixed 60,000-CAL reserve with user-paid FUEL and no top-up. The nine-service deployment uses disk-backed committee application state and BlockStore, in-memory members and gateway, and NoSync wallets. The driver releases each withheld parent only after its compensation command is committed and its historical revision is materialized on all four replicas, so every withheld source is compensated after the 30-s public-time deadline and executes afterwards.

All 21,600 payments close. The 216 compensations (12 per 1% run, 60 per 5% run) are each followed by repayment. At the end of every run no obligation is Open or Repaired, the gap is zero, net Spent and Reserved

TABLE S17

CLOSING AUDIT OF THE COMPENSATION AND REPAYMENT MATRIX. EACH COLUMN IS REPRODUCED BY ALL THREE SEEDS; EACH RUN HAS 2,400 PAYMENTS. RESERVE VALUES ARE CAL, FEE VALUES ARE IN THOUSANDS OF FUEL, AND FEE CAPS SUM THE PER-PAYMENT MAXIMA.

Withheld parents	0%	1%	5%
Compensations / repayments	0	12	60
Paid / repaid CAL	0	1,200	6,000
Final reserve	60,000	60,000	60,000
Unclosed / net Spent	0	0	0
Total fee caps	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

are zero, and the reserve is back at 60,000 CAL (Table S17). The lowest sampled reserve is 59,800–59,900 CAL at 1% and 59,600 CAL at 5%. It reflects the driver’s release schedule rather than a reserve-sizing result, and Fig. S4 shows the three 5% trajectories. Run-level receipt P50s for normal child payments are 1.185–1.251 ms. Measured from the deadline, the committee commit, which in this build carries the representation, is observed at a median of 1.003–1.254 s, and physical installation of the revised body on all four replicas at a median of 1.257–1.761 s (P95 up to 2.015 s); both include 250-ms observation polling. Each normal payment spends 94 FUEL and each compensation adds 5, and rewards, burn, and refunds sum to the fee caps with zero Held.

Ledger-level regression covers arrival orders directly: all 24 arrival orders of a four-hop chain, with and without due compensation, under three issuer sequences give 144 trajectories, and 48 split-merge trajectories vary all arrival orders of a 40/60 split, two branches, and their merge under two compensation modes. Every step in the 192 trajectories conserves the independently computed asset projection. Every trajectory ends with each issuer’s reserve restored, Spent and Reserved zero, Paid = Rec, and the consumed inputs still consumed.

Two fourteen-service controls separate permanent loss from delayed repayment. When the parent never arrives, the grandchild keeps its 100 CAL and the responsible issuer’s reserve stays 100 CAL lower. When a cross-organization chain arrives grandchild first, the intermediate source closes its own direct obligation, and the root issuer’s compensated 100 CAL is repaid when the root later executes. The four committee states agree in both cases, and an absent parent’s own fee and work reservations can remain outstanding.

Three paired fourteen-service runs on the initial baseline build, before the batch-result tag separation (4,000 payments at 100 payments/s each), end with identical owner holdings and reserve balances in normal and withheld-source runs, consistent with the CAL delay-neutrality theorem in the main paper. Concurrent compensation raises receipt P95 by a factor of 1.98–2.21, and the pairs do not isolate the responsible stage.

A storage control with NoSync wallets isolates member commits: receipt P50 is 1.0 ms with NoSync members and 43–52 ms with synchronous commits.

An earlier-build capital series with one run per condition and no compensation shows that external additions of 10,800–19,700 CAL sustain 300-s runs of 6,000 two-hop units from a 3,600-CAL start, whereas a fixed 3,600-CAL control closes 46 of 6,000 units.

C. Continuation Across Registered Organizations

To measure the effect of communication distance, we use a two-organization suite on an archived build recorded in the supplement. Its 24 runs complete 342,000 payments, with two wallet processes holding 32 identities each across 64 lanes and each organization supplying 50 payments/s. The latency comparison uses ten-hop same- and cross-organization workloads at 0, 20, and 100 ms injected HTTP RTT, three repetitions per condition. Each proxy direction adds half the RTT, and intra-organization certification and committee P2P receive no delay injection.

For the common first-120-s cohort, the median of three run-level cross-organization ACK P50s is 1.632, 21.658, and 101.967 ms, respectively, and same-organization values span 1.615–1.744 ms (Fig. S8). Cross-organization ACK P50 tracks the injected RTT, with a measured excess of 1.6–2.0 ms. An ACK includes the return path, but a recipient

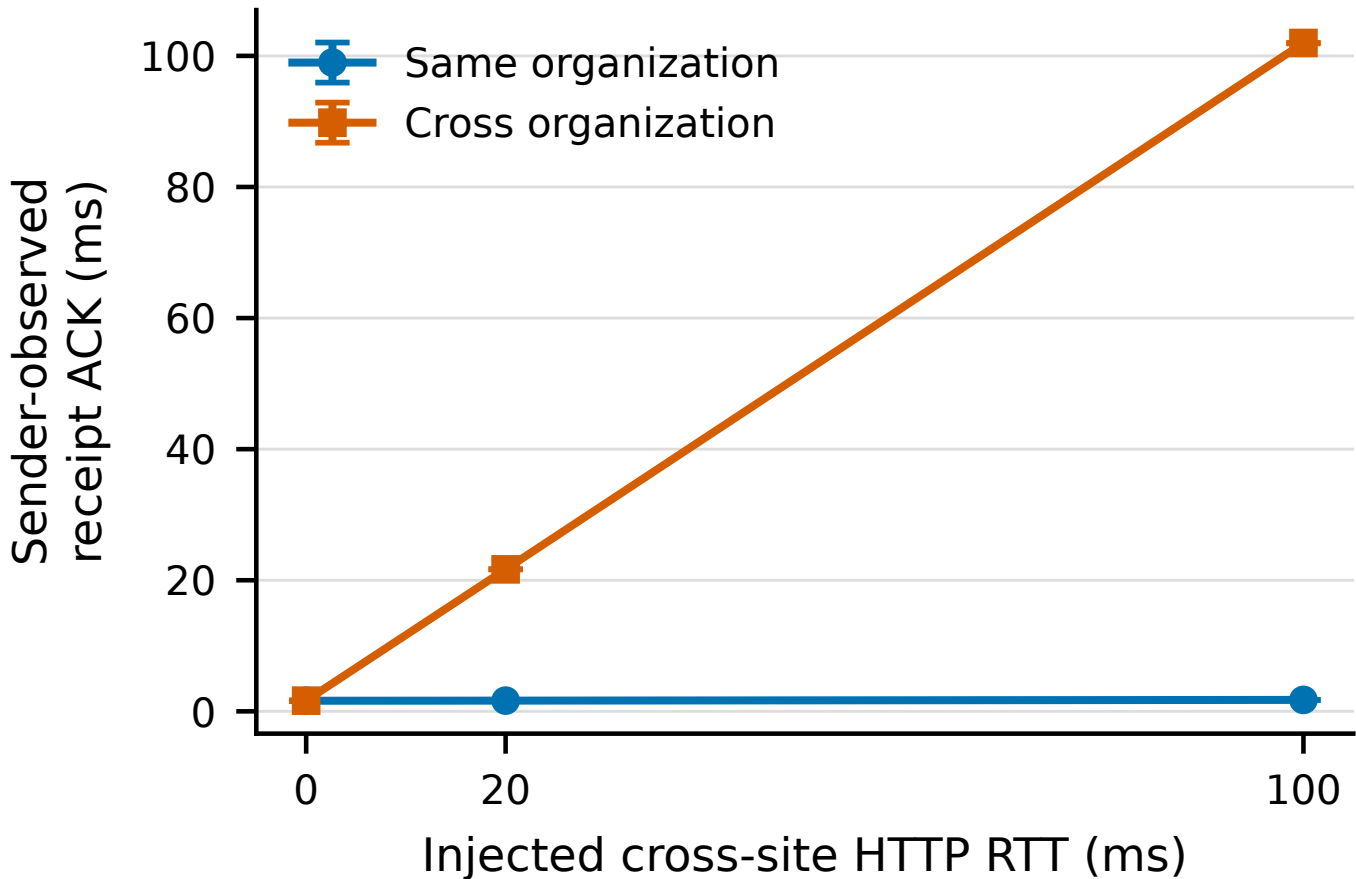


Fig. S8. ACK latency (ms) versus injected cross-site HTTP RTT (ms). Each point is the median of three run-level P50s for the first 120-s cohort; error bars span those P50s' minimum and maximum. Aggregate offered rate is 100 payments/s. Committee P2P is undelayed.

may start its successor on acceptance without waiting for the ACK to reach the previous sender. At 100-ms injected RTT in the cross-organization workload, three 300-s runs each complete 30,000 payments; pending peaks are 106–110 and drain without sustained tail growth. The experiment isolates HTTP path delay within one physical host.

D. Operating Capacity and Service Availability

Independent-input throughput (baseline build). Three 30,000-payment runs target 2,000 payments/s on the nine-service in-memory configuration with NoSync wallets and authorized organization-paid FUEL. Member processes use a 200% Go GC target and 16 runtime processors. All payments complete and the four committee states agree. Completion rates including drain are 1,750.27, 1,820.91, and 1,887.31 payments/s over 15.9–17.1 s, with receipt P50s of 2.646, 2.842, and 2.824 ms and P95s of 47.9, 49.1, and 48.0 ms. The driver recorded 6,551–7,570 payments that had to wait for a fast-receipt admission slot, and the P95 dispatch lag was 179–229 ms, so the offered load was not uniform at 2,000 payments/s. These are short-run completion measurements on the baseline build. They are not remeasured for the decision-first build, whose cost on ordinary payments is tested by the six-run comparison above. The supplement identifies the build and duration of the 150,000-payment series.

On archived builds, redundant certification and evidence replicated by INSTALL sustain service under the specified failures. Nine runs at 200 payments/s complete 216,000 payments with one member delayed or paused for 30 s. Gateway-loss controls in which one member holds a complete copy replicated by INSTALL show backup submission 2.503–2.508 s after INSTALL. Conflict controls produce neither dual conflicting QCs nor repeated principal/fee effects, while partial approvals remain a capacity cost.

The multi-identity measurements in Section S5 report per-payment CPU and HTTP-payload costs; 2,048 identities share clients. Historical memory growth is reported separately from backlog. The same section gives an LND regtest reference, whose completion event, persistence, and workload concurrency differ from the Next measurements.

S7. PER-RUN EXPERIMENTAL RECORDS

TABLE S18

ARCHIVED ONE-ORGANIZATION OPERATING-POINT RUNS. EACH RUN SENDS AND COMPLETES 150,000 PAYMENTS. INJECTION AND COMPLETION ARE IN PAYMENTS/S; RECEIPT QUANTILES ARE IN MS. COMPLETION INCLUDES FINAL BACKGROUND DRAIN. PEAK IS THE MAXIMUM NUMBER OF PAYMENTS WITH UNFINISHED BACKGROUND PROCESSING; LIMIT HITS COUNTS TRIGGERS OF THE 4,096-UNFINISHED-PAYMENT LIMIT.

Run	Target	Injection	Completion	P50	P95	Peak	Limit hits
1	2000	1,964.53	1,948.66	2.521	50.499	2159	0
2	2000	1,969.65	1,950.45	2.566	50.761	2205	0
3	2200	2,127.70	2,103.63	4.538	69.463	2655	0
4	2200	2,127.15	2,106.29	4.553	69.945	2548	0
5	2400	2,206.95	2,174.68	7.174	89.376	4096	1635

TABLE S19

BASELINE SOURCE-RECOVERY RUNS. EACH OFFERS 1,200 TWO-HOP UNITS (2,400 PAYMENTS) OVER 60 SECONDS. RECEIPT QUANTILES FOR NORMAL CHILD PAYMENTS ARE IN MS; ELAPSED AND DEADLINE-TO-ALL-REPLICA-INSTALLATION OBSERVATION MEDIAN ARE IN SECONDS. THE LATTER INCLUDES 250-MS POLLING. EVERY RUN ENDS WITH A 60,000-CAL RESERVE, ZERO NET EXPENDITURE, AND ZERO GAP.

Seed	Withheld	Paid/recovered	Elapsed	Child P50	Child P95	Install P50
23	0%	0/0	61.540	1.251	1.620	–
23	1%	12/12	90.225	1.185	1.432	1.759
23	5%	60/60	90.096	1.191	1.536	1.257
37	0%	0/0	61.595	1.199	1.559	–
37	1%	12/12	91.935	1.190	1.479	1.509
37	5%	60/60	91.530	1.192	1.426	1.257
59	0%	0/0	61.550	1.191	1.461	–
59	1%	12/12	90.595	1.193	1.452	1.761
59	5%	60/60	90.346	1.194	1.494	1.507

TABLE S20

ARCHIVED FINITE-RESERVE RUNS. ONE 300-S RUN PER CONDITION; CAPITAL VALUES ARE CAL. CLOSED COUNTS REFER TO TWO-HOP UNITS. ADAPTIVE INCREASES ARE ACTUAL TRANSFERS FROM AN EXTERNAL FUNDING ACCOUNT. MEAN IS THE TIME-AVERAGE ORGANIZATION CAL BALANCE OBTAINED BY INTEGRATING BALANCE SAMPLES OVER THE 300-S INPUT WINDOW.

Condition	Initial	Added	Final	Mean	Closed
Constant / fixed low	3,600	0	3,600	3,600.0	46/6000
Constant / fixed high	14,400	0	14,400	14,400.0	6000/6000
Constant / adaptive	3,600	10,800	14,400	13,801.8	6000/6000
Step / fixed high	21,600	0	21,600	21,600.0	6000/6000
Step / adaptive	3,600	18,000	21,600	14,078.7	6000/6000
Delayed / fixed high	28,800	0	28,800	28,800.0	6000/6000
Delayed / adaptive	3,600	19,700	23,300	20,611.2	6000/6000

REFERENCES

- [1] V. Shoup, “Practical Threshold Signatures,” in *Advances in Cryptology—EUROCRYPT*, 2000, pp. 207–220. [Online]. Available: <https://www.iacr.org/archive/eurocrypt2000/1807/18070209-new.pdf>

TABLE S21

SYNCHRONOUS-MEMBER CONTINUATION CONTROLS: 100 HOPS PER RUN, NINE DISK-BACKED SERVICES, USER-PAID FUEL, AND NoSYNC WALLETS. DURATIONS END AT THE LAST CERTIFIED RECEIPT AND ARE IN SECONDS. N/S ORDER APPLIES TO THE TWO FAST MODES; THE SYNCHRONOUS WAITING RUN FOLLOWS EACH PAIR.

Pair	Order	NoSync fast	Sync fast	Sync wait
1	N/S	0.681127	4.492812	64.969480
2	S/N	0.705604	4.309984	65.193522
3	N/S	0.636821	4.470844	65.012947
Median		0.681127	4.470844	65.012947